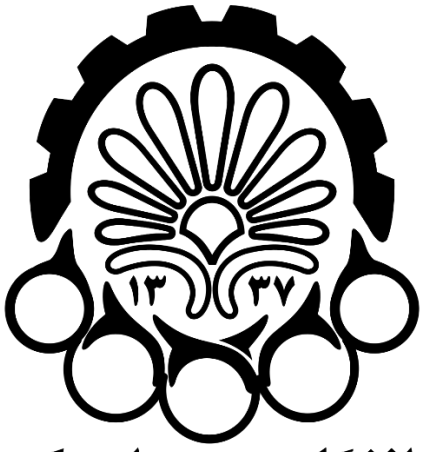


دانشگاه صنعتی امیرکبیر

(پلی تکنیک تهران)

گزارش پروژه اول مبانی هوش مصنوعی

نویسنده: کیانا شرافتی ۴۰۱۳۱۰۲۲

تحلیل جامع پروژه اول: جستجو در فضای حالت (سوکوبان)

۱. تشریح فرآیند پیاده‌سازی

وظیفه اصلی پروژه، پیاده‌سازی الگوریتم‌های جستجو برای حل پازل سوکوبان با قوانین هزینه متغیر بود.

۱,۱. مدل‌سازی فضای حالت (State)

بخش کلیدی در هر مسئله جستجو، تعریف دقیق «حالت» است. در کد `main.py`، حالت به صورت زیر تعریف شده است:

```
@dataclass(frozen=True)
class State:
    player: Tuple[int, int]
    boxes: FrozenSet[Tuple[int, int]]
```

- **تحلیل:** این ساختار به درستی وضعیت فیزیکی تخته (مکان بازیکن و جعبه‌ها) را مدل می‌کند. استفاده از `frozen=True` و `FrozenSet` آن را قابل هش (Hashable) کرده و امکان استفاده در `Set` و `Dict` (برای `visited` یا `best_g`) را فراهم می‌کند.
- **تناقض کلیدی با الزامات پروژه:** الزامات پروژه (PDF) به صراحت بیان می‌کنند که هزینه‌ها **پویا** هستند و به «سیب» (باعث کاهش هزینه هل دادن به ۱ برای ۱۰ حرکت) و «سم» (افزایش دائمی +۳ به تمام حرکات) بستگی دارند.
  - مدل‌سازی صحیح برای الگوریتم‌های آگاه از هزینه (UCS, A*) نیازمند آن بود که وضعیت سیب و سم بخشی از خود State باشد، مثلاً:

State(player, boxes, apple_buff_steps, poison_active)

  - **مدل‌سازی فعلی:** `State` فعلی فاقد این اطلاعات است. این موضوع اساسی‌ترین دلیل برای رفتارهای "غیرمنتظره" الگوریتم‌ها است که در بخش ۳ به آن می‌پردازیم.

۲,۱. تابع جانشین (Successors) و هزینه‌ها

تابع `SokobanProblem.successors` مسئول تولید حرکات مجاز است.

- این تابع به درستی ۴ جهت را بررسی کرده و دو نوع حرکت را تولید می‌کند:

۱. **حرکت ساده:** اگر مقصد آزاد باشد، هزینه **۱,۰** برمی‌گرداند.
۲. **هل دادن جعبه:** اگر مقصد جعبه باشد و پشت آن آزاد باشد، هزینه **۵,۰** برمی‌گرداند.

- **نقد:** این هزینه‌ها **ایستا (Static)** هستند. الگوریتم‌های جستجو (UCS و A*) از این هزینه‌ها برای محاسبه g(n) استفاده می‌کنند، نه از هزینه‌های پویا و واقعی بازی که در PDF توضیح داده شده‌اند. منطق محاسبه هزینه واقعی (با سیب و سم) تنها در تابع `Game._compute_history_cost` (که فقط هزینه بازی دستی را نمایش می‌دهد) پیاده‌سازی شده و در دسترس الگوریتم‌های هوشمند نبوده است.

۳,۱. ساختارهای داده الگوریتم‌ها

ساختارهای داده استاندارد برای هر الگوریتم به درستی انتخاب شده‌اند:

- **DFS**: از یک `list` پایتون به عنوان **پشته (Stack)** استفاده می‌کند.
- **BFS**: از `collections.deque` به عنوان **صف FIFO (Queue)** استفاده می‌کند.
- **UCS, Greedy, A***: هر سه از `heapq` به عنوان **صف اولویت (Priority Queue)** برای مدیریت کارآمد گره‌ها بر اساس اولویت (g, h, یا f) استفاده می‌کنند.

۲. تحلیل تابع هیوریستیک (Heuristic)

برای الگوریتم‌های Greedy و A*، یک تابع هیوریستیک `h_greedy` پیاده‌سازی شده است.

۲,۱. تعریف و نحوه محاسبه

این هیوریستیک، «مجموع حداقل فاصله منهتن» نام دارد.

- **فرآیند:** به ازای هر جعبه، فاصله منهتن ($|x_1 - x_2| + |y_1 - y_2|$) تا نزدیک‌ترین هدف (Goal) محاسبه می‌شود.
- **نتیجه:** $h(n)$ برابر است با مجموع این فواصل حداقل برای تمام جعبه‌ها.

۲,۲. ارزیابی: خوش‌بینانه بودن (Admissibility)

این هیوریستیک خوش‌بینانه (Admissible) است.

- **چرا؟** هیوریستیک خوش‌بینانه هرگز هزینه واقعی را بیشتر از واقعیت تخمین نمی‌زند.
  - هزینه واقعی ($h^*(n)$) هزینه حل یک «تطابق کامل» (Perfect Matching) است؛ یعنی رساندن هر جعبه به یک هدف منحصربه‌فرد.
  - هیوریستیک ما ($h(n)$) ساده‌سازی شده است و اجازه می‌دهد چند جعبه، یک هدف یکسان را به عنوان نزدیک‌ترین مقصد خود انتخاب کنند.
  - هزینه این تطابق ساده‌سازی‌شده (که مجاز است) همیشه کمتر یا مساوی هزینه تطابق کامل واقعی (که سخت‌گیرانه‌تر است) خواهد بود.
  - از آنجایی که $h(n) \le h^*(n)$، هیوریستیک admissible است و A* با استفاده از آن، تضمین می‌کند که مسیر بهینه را پیدا کند.

۳,۲. دلیل انتخاب

این هیوریستیک توازن خوبی بین «دقت» و «سرعت محاسبه» است. محاسبه آن ($O(\text{Targets} \times \text{Boxes})$) بسیار سریع‌تر از هیوریستیک‌های دقیق‌تری مانند الگوریتم مجارستانی (Hungarian) است که محاسبه آن‌ها ($O(\text{Boxes}^3)$) در هر گره جستجو، زمان کل حل را به شدت افزایش می‌داد.

۳. تحلیل نتایج و رفتار الگوریتم‌ها

نتایج به‌دست‌آمده از اجرای الگوریتم‌ها روی لول‌های ۴ و ۵ به شرح زیر است:

الگوریتم	Level	هزینه (Cost)	زمان (ms)
DFS	۴	۶۳۶۱	۱۶۵.۴۵۶
BFS	۴	۱۷۹	۲۲۶.۱۲۷
Greedy	۴	۲۳۱	۳۱
UCS	۴	۱۷۱	۳۶۱.۴۳۸
A*	۴	۱۷۱	۳۳۲.۸۱۶
DFS	۵	۱۴۶۴۱	۳۸۳.۴۸۰
BFS	۵	۳۰۹	۲۰۰.۶۸۱
Greedy	۵	۳۹۱	۲۲۵
UCS	۵	۲۹۷	۳۳۱.۲۸۷
A*	۵	۲۹۷	۳۲۱.۳۸۵

۱,۳. تحلیل بهینگی (Cost)

۱. UCS و A* (بهینه):

همانطور که تئوری پیش‌بینی می‌کند، هر دو الگوریتم UCS و A* (به دلیل هیوریستیک admissible) دقیقاً مسیر بهینه را پیدا کردند (هزینه ۱۷۱ در لول ۴ و ۲۹۷ در لول ۵). این نشان می‌دهد که هر دو الگوریتم، بهینگی بر اساس هزینه (Cost Optimality) را تضمین می‌کنند.

۲. BFS (بهینه عمق):

BFS کوتاه‌ترین مسیر بر اساس تعداد گام‌ها (Depth Optimality) را پیدا می‌کند، نه هزینه. هزینه‌های آن (۱۷۹ و ۳۰۹) کمی بالاتر از بهینه است. این بدان معناست که BFS مسیری با تعداد حرکات کمتر پیدا کرده که شامل هل دادن‌های پرهزینه (۵,۰) بیشتری نسبت به مسیر بهینه A*/UCS بوده است.

۳. **DFS و Greedy (غیر بهینه):**

- **DFS** به عنوان یک جستجوی کور، اولین مسیری که پیدا کرده را برگردانده که از نظر هزینه (۶۳۶۱ و ۱۴۶۴۱) بسیار نامطلوب و طولانی است.
- **Greedy** فقط هیوریستیک را دنبال کرده و هزینه‌های گذشته (g(n)) را نادیده گرفته است. این باعث شده به سرعت به جوابی برسد (۳۱ میلی‌ثانیه)، اما جوابی که از نظر هزینه ضعیف است (۲۳۱).

۳,۲. تحلیل کارایی (Time)

۱. Greedy (سریع‌ترین):

Greedy با اختلاف زیاد سریع‌ترین است. با تمرکز «حریصانه» بر کاهش فاصله تا هدف، بخش بسیار کوچکی از فضا را کاوش می‌کند.

۲. A* در برابر UCS (قدرت هیوریستیک):

در هر دو لول، A* به طور قابل توجهی سریع‌تر از UCS عمل کرده است (حدود ۸ درصد در لول ۴ و ۳ درصد در لول ۵). UCS (که مشابه دایجسترا عمل می‌کند) کورکورانه در تمام جهات بر اساس هزینه g(n) گسترش می‌یابد. اما A* با استفاده از h(n)، جستجو را به سمت هدف «هدایت» می‌کند و گره‌های بسیار کمتری را نسبت به UCS بسط می‌دهد تا به جواب بهینه یکسان برسد.

۳. پیچیدگی و مقیاس‌پذیری:

با افزایش پیچیدگی از لول ۴ به ۵، زمان اجرای DFS به شدت افزایش یافت (بیش از دو برابر)، که نشان‌دهنده عمق زیاد درخت جستجو است. زمان A* و UCS نیز افزایش یافت، که نشان‌دهنده رشد نمایی فضای حالت است.

۳,۳. تحلیل رفتار غیرمنتظره: خوردن سم

مشاهده: طبق گفته شما، در لول آخر (و احتمالاً لول‌های دیگر) تمام الگوریتم‌ها مسیری را انتخاب کردند که شامل «خوردن سم» (عبور از کاشی 'p') بود.

توضیح (چرا این اتفاق اجتناب‌ناپذیر بود؟):

این رفتار، یک «اشکال» در الگوریتم‌ها نیست، بلکه نتیجه مستقیم مدل‌سازی ناقص مسئله در پیاده‌سازی فعلی است.

۱. **قانون واقعی:** PDF می‌گوید سم هزینه تمام حرکات آینده را +۳ می‌کند. یک عامل هوشمند واقعی باید از آن (حتی به قیمت طولانی‌تر شدن مسیر) اجتناب کند.
۲. **مدل ارائه‌شده به الگوریتم:** همانطور که در بخش ۱,۱ و ۱,۲ توضیح داده شد، نه `State` (که فاقد `poison_active` است) و نه `successors` (که هزینه ثابت ۱,۰ برمی‌گرداند) هیچ درکی از «سم» ندارند.
۳. **نتیجه:** از دیدگاه هر پنج الگوریتم (A*,DFS, BFS, UCS, Greedy)، کاشی 'p' (سم) هیچ تفاوتی با کاشی ' ' (زمین خالی) ندارد. هر دو صرفاً یک خانه قابل عبور با هزینه ۱,۰ هستند.
۴. **چرا همه آن را خوردند؟** الگوریتم‌ها کوتاه‌ترین یا کم‌هزینه‌ترین مسیر را بر اساس مدلی که به آن‌ها داده شده پیدا کردند. اگر کوتاه‌ترین مسیر (بر اساس هزینه‌های ۱,۰ و ۵,۰) به طور اتفاقی از روی کاشی 'p' عبور می‌کرد، الگوریتم‌ها هیچ دلیلی برای اجتناب از آن نداشتند. آن‌ها به طور کامل و بهینه، مسئله‌ی اشتباهی را حل کردند.

برای حل این مشکل، می‌بایست `State` گسترش می‌یافت و `successors` هزینه‌های پویا و واقعی را بر اساس قوانین سیب و سم محاسبه می‌کرد.

۴ .نتایج تجربی و مستندات بصری

برای ارزیابی عملی الگوریتم‌ها، تمام پنج روش جستجو بر روی تمامی مراحل اجرا شدند. در سطوح اولیه بازی (۱ تا ۳)، به دلیل سادگی نسبی نقشه‌ها و کوچک بودن فضای حالت، اکثر الگوریتم‌ها (به جز DFS) به سرعت به جواب می‌رسیدند و تفاوت عملکردی فاحشی میان آن‌ها مشاهده نمی‌شد.

با این حال، تفاوت واقعی در کارایی و بهینگی الگوریتم‌ها در مراحل پیچیده‌تر، یعنی **لول ۴ و ۵**، آشکار می‌شود. در این مراحل، فضای حالت به شدت بزرگتر بوده و چالش‌های بیشتری (مانند بن‌بست‌های متعدد و فواصل طولانی‌تر) وجود دارد.

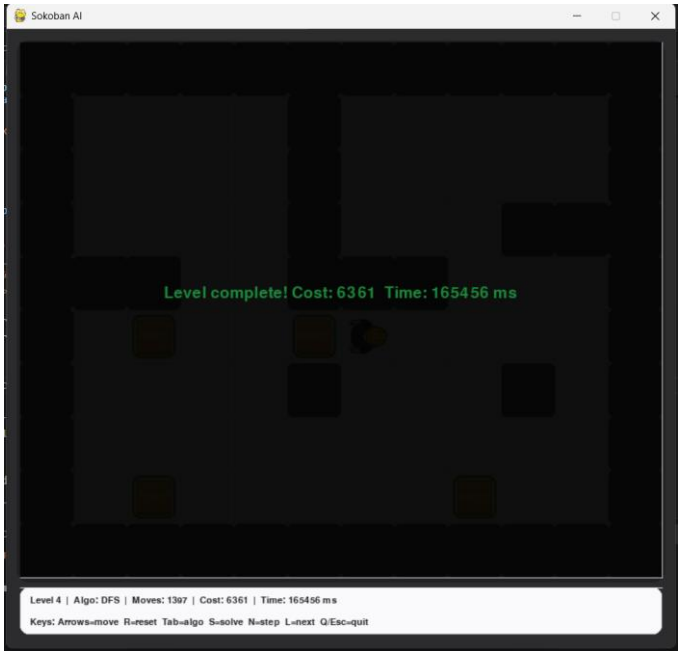
تصاویر زیر (که به عنوان مستندات ضمیمه شده‌اند) نتایج نهایی و آمار کلیدی هر الگوریتم را در این دو لول نمایش می‌دهند:



BFS Level 4



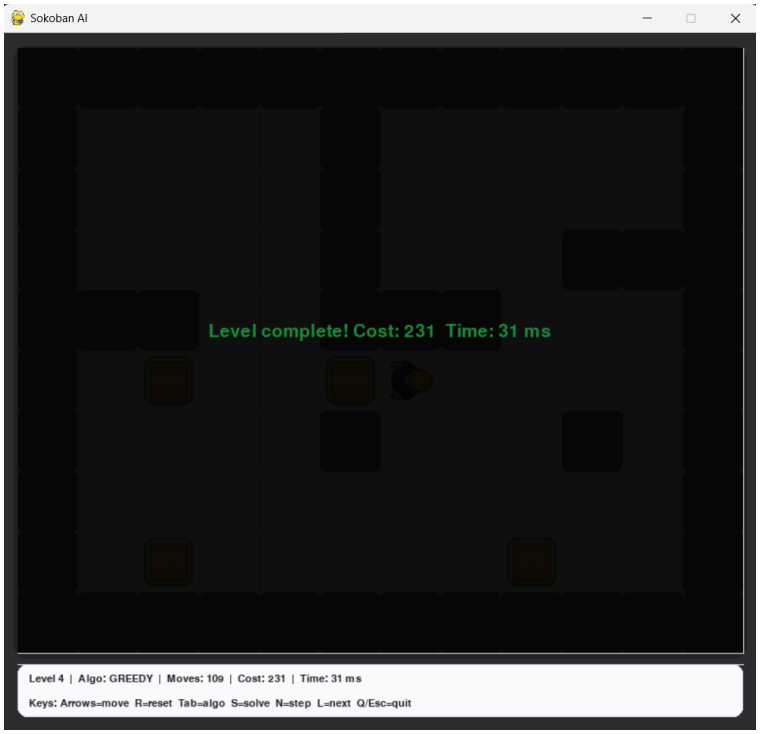
BFS Level 5



DFS Level 4



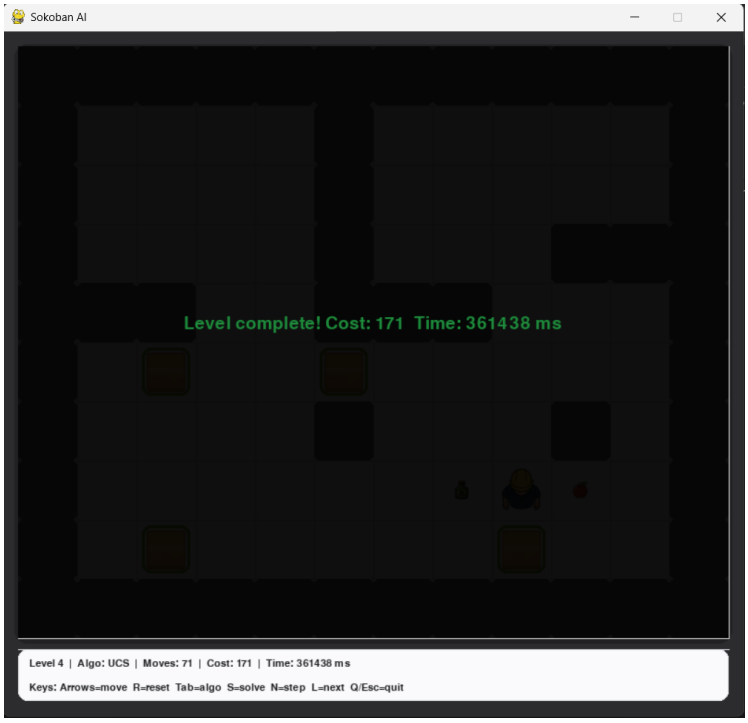
DFS Level 5



Greedy Level 4



Greedy Level 5



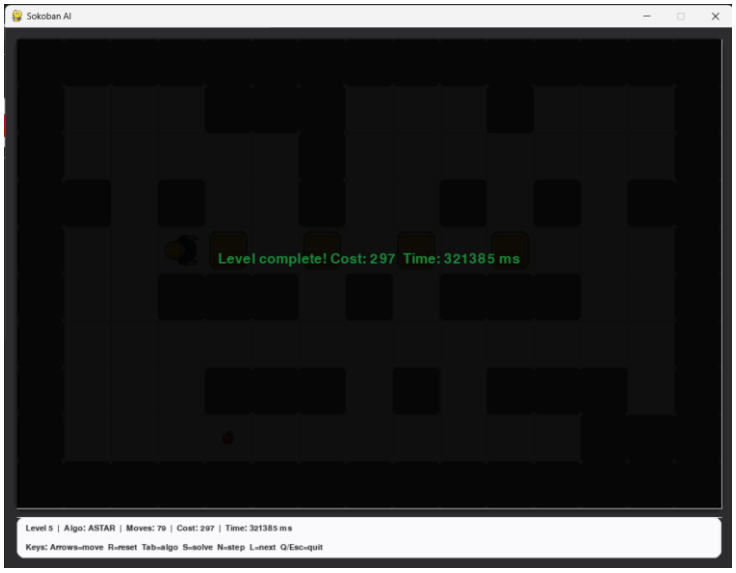
UCS Level 4



UCS Level 5



A* Level 4



A* Level 5